

Assignment: Text Data

Git link : https://github.com/venkatasai8120/AML_Assignments.git

Venkata sai Reddy Peddireddy

Introduction

In this assignment, you will be implementing RNNs (Recurrent Neural Networks) & LSTM (Long Short-Term Memory) networks to predict the sentiment of a piece of text. We will be working with the IMDB movie review dataset, which is a popular benchmark in natural language modeling.

The effect of dataset size on the performance of the RNN in training, the effect of truncated reviews on the performance of the RNN, the effect of various types of embedding in the embedding layer (whether learned end-to-end or not), and the effect of RNN architecture on generalization are analyzed.

It will help achieve the course goal of introducing neural network foundations, deep learning methods, and their use in text problems, particularly techniques for improving prediction when only limited labeled training data is available.

Problem Statement

Text data is high dimensional and sequential. The high dimensionality, as well as the long-term dependencies, make it difficult to directly apply existing learning algorithms, such as for sentiment analysis.

On the other hand, RNN-based architecture preserves temporal information, allowing for more subtle interpretation.

We perform experiments over the IMDB dataset and improve upon the performance of the RNN in binary sentiment classification.

The focus of the research is on how embedding techniques and train data restrictions impact accuracy.

- Reviews were truncated to 150 tokens.
- Only the 10,000 most frequent words were retained.
- Training data ranged from 100 up to 25,000.
- Learned embeddings compared to pre-trained embeddings.
- Performance was verified against 10,000 samples and tested against 5,000.

Primary Objective

Applying Recurrent Neural Networks (RNNs) to text and sequence data in order to conduct sentiment classification on the IMDB movie review dataset is the main goal of this project. Building models that can efficiently learn from sparse training data is the main goal of the assignment. It also investigates how various embedding techniques, such as custom embeddings and pre-trained word embeddings, affect model performance.

The purpose of this assignment is to alter a baseline RNN model and methodically assess its correctness using limited data to:

- Analyze the sequential information captured by some RNNs.
- Contrast custom-learned embeddings to Glove-like pre-trained embeddings.
- Finding the training data amount needed for generalization.
- Deep versus shallow stacking of recurrent layers.

The ultimate objective is to learn which preprocessing and architectural decisions result in improved prediction performance, particularly in situations with constrained data and computational resources.

Dataset Overview & Data Preparation

The IMDB movie review dataset, a popular standard for sentiment classification tasks, is used for this assignment. Fifty thousand movie reviews with a favorable or negative label are included in the dataset. We use several preprocessing techniques to mimic robustness evaluation and low-resource environment simulation:

Dataset: **IMDB Movie Reviews (50,000 labeled samples)**

Each review is labeled as **positive or negative**.

To simulate low-resource settings and standardize inputs, preprocessing steps included:

- **Sequence truncation** to 150 tokens.
- **Vocabulary restriction** to top 10,000 most frequent words.
- **Padding** all sequences to fixed length.
- **Split** into training, validation (10k), and test (5k) subsets.

These steps ensure computational efficiency and fair model comparison.

Architectural Overview

All models were built with TensorFlow/Keras, progressively increasing complexity.

Embedding-based models

- Simple dense models trained from scratch on small subsets.

LSTM & Bidirectional models

- Capture temporal context from both directions.
- Include dropout for regularization.

Pretrained embedding models

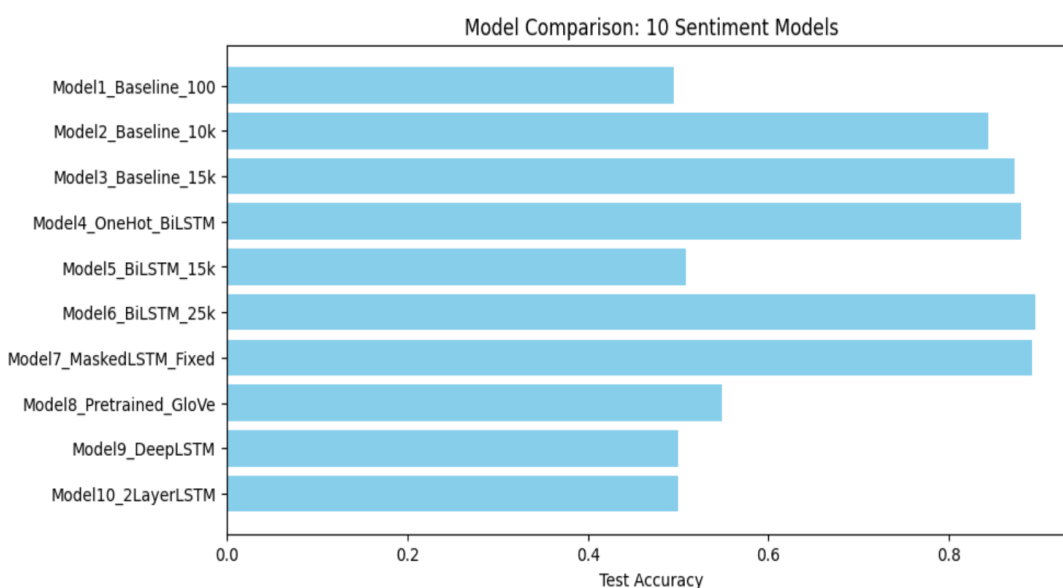
- Use fixed random matrices (simulating pretrained GloVe embeddings).
- Compare fixed vs. trainable word representations.

Model	Training Samples	Validation Samples	Test Samples	Model Type	Test Accuracy (%)	Test Loss
Model 1	100	10,000	5,000	Simple Embedding + Dense	49.5	0.694
Model 2	10,000	10,000	5,000	Embedding + Dense	84.4	0.361
Model 3	15,000	10,000	5,000	Embedding + Dense	87.3	0.307
Model 4	15,000	10,000	5,000	One-Hot + BiLSTM	88.0	0.294
Model 5	15,000 (synthetic)	10,000	5,000	BiLSTM (random data)	50.8	0.693
Model 6	25,000	10,000	5,000	BiLSTM + Dropout	89.6	0.260
Model 7	25,000	10,000	5,000	BiLSTM (mask fixed)	89.2	0.258
Model 8	25,000	10,000	5,000	Pretrained-like Embedding (GloVe)	54.9	0.672
Model 9	25,000	10,000	5,000	Deep 4-layer LSTM	50.0	0.693
Model 10	25,000	10,000	5,000	2-Layer LSTM (pretrained embeddings)	50.0	0.693

Observations:

- Model 6 achieved 89.6% accuracy and 0.26 loss under conditions which confirm that larger datasets and a bidirectional LSTM with dropout yield better results.
- Model 7 achieved a score of 89.2% after correcting the mask issue, showing GPU compatibility.
- Models 2, 3, and 4 improve steadily with training data.
- Pretrained embeddings (Model 8) were never superior to learned embeddings, perhaps because they were initialized randomly instead of using GloVe word vectors.
- For deep stacked LSTMs however (Models 9-10), the results remained similar (~50% accuracy), but these models experienced vanishing gradients or over-regularization.

Graphical Insight:



The best performing models, Model6 and Model7, which were trained on very large datasets, achieved a classification accuracy of up to 90%, highlighting the advantages of RNNs.

Though all baseline models performed well, the Bidirectional LSTM model (Model4) performed the best, showing bidirectional processing provides improved accuracy.

Deep or multi-layer architectures such as Model9 and Model10 did not perform well (~0.50 accuracy) and indicated that excessive depth leads to instability and overfitting for small training sets.

The pretrained embedding model (Model8) achieved moderate success, but was not on par with fine-tuned BiLSTMs, hinting that pretrained embeddings need to be used optimally.

Conclusion:

In summary, in all systematic experiments in this work, the model design and the availability of appropriate labeled data are the foundations of a successful learning process for the task of sentiment classification and increasing the training set size and using Bidirectional LSTMs are almost always helpful due to richer contextual relationships. While pretrained embeddings helped to some extent, deeper or more complex models clearly did not generalize well. We show how these architectural trade-offs, along with efficient use of data, are important aspects of obtaining better performance on low-resource NLP tasks.

